

Distributed Taxi Trip Analytics

Using Apache Hadoop, HDFS and Python MapReduce

Big Data Essentials — Individual Practical Case Study

NYC Taxi & Limousine Commission Yellow Taxi Trip Records, January–March 2026

Records processed	11,077,209 raw trips
Records analysed	10,493,916 after cleaning
Raw dataset size	1.10 GB CSV (converted from 182 MB of Parquet)
Platform	Apache Hadoop 3.5.0, pseudo-distributed, OpenJDK 21
Processing model	Hadoop Streaming with Python 3.11 mappers and reducers
MapReduce jobs executed	15 pipeline + 3 benchmark (all succeeded)
Total cluster time	3,625 seconds across 18 YARN applications

*Every figure in this report is the output of a MapReduce job that actually ran.
Job logs, Hadoop counters and YARN application records are in evidence/.*

Table of Contents

1. Introduction.....	3
2. Business Problem.....	3
3. Dataset Description.....	4
4. Hadoop Environment.....	5
5. HDFS Design.....	7
6. Data Cleaning.....	8
7. MapReduce Design.....	10
8. Mapper and Reducer Implementation.....	12
9. Analytical Results.....	14
10. Multi-Stage MapReduce.....	22
11. YARN Analysis.....	24
12. Performance Comparison.....	26
13. Business Insights.....	27
14. Limitations.....	29
15. Conclusion.....	29
16. References.....	30

Figures 9.1–9.7 appear in Section 9; code listings in Sections 3, 6, 8 and 10; cluster, HDFS and YARN evidence extracts in Sections 4, 5, 10 and 11.

1. Introduction

This report documents the design, implementation and execution of a Hadoop-based analytics solution for New York City yellow-taxi trip records. The work was carried out on a live single-node pseudo-distributed Apache Hadoop 3.5.0 cluster: HDFS stores the dataset, YARN schedules the work, and Hadoop Streaming runs Python mapper and reducer programs across the cluster.

The dataset comprises three consecutive monthly releases from the NYC Taxi & Limousine Commission — January, February and March 2026 — totalling **11,077,209 trip records** and 1.10 GB of line-oriented CSV. This satisfies the assignment requirement of two-to-three monthly datasets and comfortably exceeds the five-million-record threshold.

15 MapReduce jobs make up the pipeline: one two-phase cleaning job, nine analytical jobs covering the required analyses, and five Top-N jobs forming the multi-stage workflow. A further 3 benchmark jobs were run for the performance comparison in Section 12, giving 18 YARN applications in total. All completed with exit status zero. Their application identifiers, counters and elapsed times appear throughout this report and in full in the evidence directory.

1.1 What this report argues

Three findings run through the analysis and are worth stating at the outset, because each one shaped a decision in the implementation.

- **Demand is remarkably flat across the working day and concentrated in a very small part of the city.** The busiest hour (18:00, 696,255 trips) carries only 7.8 times the volume of the quietest (04:00), and there is no sharp morning commuter spike at all. Meanwhile the top ten pickup zones out of 262 account for 33.4% of all trips.
- **The most frequent routes are not the most profitable ones.** Only 7 of the top 20 routes by trip count also appear in the top 20 by revenue. Short intra-Manhattan hops dominate volume; airport runs dominate money. This distinction is invisible in a trip-count analysis and is the single most actionable result in the report.
- **At this scale, on this hardware, Pandas beats Hadoop by a factor of 18.** That is the honest measured result, and Section 12 explains precisely why it is the expected one, where the crossover lies, and what Hadoop is actually buying.

1.2 Learning objectives addressed

Objective	Where it is demonstrated
Load and organise a large dataset in HDFS	Section 5 — directory design, block layout, fsck and dfsadmin evidence
Implement Python mapper and reducer programs	Sections 7 and 8 — 22 programs, key-value design explained
Execute Hadoop Streaming MapReduce jobs	Section 9 — all nine analyses, with counters
Understand the Shuffle and Sort stage	Section 7.2 — and the measured combiner effect in 7.4
Perform multi-stage MapReduce processing	Section 10 — five stage-2 jobs consuming stage-1 HDFS output
Use YARN to monitor MapReduce applications	Section 11 — application states, containers, resource usage
Analyse and interpret large-scale results	Sections 9 and 13
Compare conventional processing with Hadoop	Section 12 — a three-point scaling benchmark

2. Business Problem

The company is a transportation analytics provider. Its clients — fleet operators, city regulators and mobility planners — need answers to operational questions that are simple to state and expensive to compute: when is demand highest, where does revenue actually come from, which routes are worth positioning vehicles for, how do customers pay, and how much of the data can be trusted at all.

The volume is what makes this a big-data problem rather than a spreadsheet problem. A single month of yellow-taxi records is roughly 3.7 million rows; a quarter is over eleven million; a full year approaches forty-five million, and the equivalent green-taxi and for-hire-vehicle feeds multiply that several times over. Any method that

requires the whole dataset to sit in one machine's memory has a hard ceiling, and that ceiling is measured explicitly in Section 12.

2.1 The analytical questions

The nine required analyses map onto the business questions as follows. Each one becomes a separate MapReduce job, and the mapping from question to key design is the substance of Section 7.

#	Analysis	Business question it answers
a	Hourly demand	When should vehicles be on the road?
b	Daily demand	How does staffing differ between weekdays and weekends?
c	Pickup locations	Where should idle vehicles wait?
d	Revenue by zone	Which neighbourhoods generate the money, not just the trips?
e	Payment behaviour	What is the revenue mix, and where do tips come from?
f	Distance bands	How does pricing behave across trip lengths?
g	Routes	Which origin–destination pairs are worth targeting?
h	Trip duration	How does time-in-traffic affect economics?
i	Anomalies	How much of the feed is unreliable, and in what way?

2.2 Why HDFS and MapReduce

HDFS provides the storage half of the answer. It splits each file into fixed-size blocks distributed across the cluster, replicates them for fault tolerance, and exposes the whole thing as one namespace. Crucially it also exposes where each block lives, which is what allows the computation to be sent to the data instead of the data to the computation.

MapReduce provides the processing half. Its restriction — a map function that sees one record at a time and a reduce function that sees one key's values at a time, with no shared state — is exactly what makes it parallelisable without coordination. The programmer never writes a lock, a thread or a partition; they choose what the key is, and the framework's Shuffle-and-Sort stage does the grouping.

3. Dataset Description

Source: the official NYC Taxi & Limousine Commission Trip Record Data portal, <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>. Three consecutive monthly Yellow Taxi files were used, together with the official Taxi Zone lookup table that maps the numeric LocationID values to zone and borough names.

File	Format	Size on disk	Records
yellow_tripdata_2026-01.parquet	Parquet	61.2 MB	3,724,889
yellow_tripdata_2026-02.parquet	Parquet	56.0 MB	3,399,866
yellow_tripdata_2026-03.parquet	Parquet	64.7 MB	3,952,451
Total (Parquet)	—	182.0 MB	11,077,209
Total (converted CSV)	CSV	1.10 GB	11,077,209

Table 3.1 — Source files. The 6.2x expansion from Parquet to CSV is the price of a line-oriented format, and is discussed below.

3.1 Schema

Each record has twenty fields. The columns that carry the analysis are marked.

#	Field	Type	Used for
0	VendorID	int32	—
1	tppe_pickup_datetime	timestamp	Analyses a, b, h — and duration is derived from 1 and 2
2	tppe_dropoff_datetime	timestamp	Analyses h, i
3	passenger_count	int64	Flagged in cleaning; see Section 6.3

#	Field	Type	Used for
4	trip_distance	double	Analyses f, i
5	RatecodeID	int64	—
6	store_and_fwd_flag	string	—
7	PULocationID	int32	Analyses c, d, g
8	DOLocationID	int32	Analysis g
9	payment_type	int64	Analysis e
10	fare_amount	double	Analyses d, e, f, h, i
11–15	extra, mta_tax, tip_amount, tolls_amount, improvement_surcharge	double	tip_amount feeds analyses d and e
16	total_amount	double	Analyses d, e, f, g, h
17–19	congestion_surcharge, Airport_fee, cbd_congestion_fee	double	—

3.2 Parquet for storage, CSV for streaming

The TLC publishes Parquet. Parquet is columnar and compressed: a query that touches three of twenty columns reads only those three columns' pages, and the three months occupy 182 MB rather than 1.1 GB. For archival storage and for engines that understand it, Parquet is unambiguously the right format.

It is nevertheless the wrong format for this assignment, for a specific mechanical reason. A Hadoop Streaming mapper is a plain process that reads *one text line at a time from standard input*. Parquet is not line-oriented; it is a block-structured binary format with a footer, and a byte offset into a Parquet file is not the start of anything. Feeding it to a streaming mapper would require a custom InputFormat written in Java — which would defeat the purpose of using Python.

The conversion is therefore a deliberate, documented trade: 6.2 times more bytes on disk in exchange for a format that HDFS can split at any newline and that a Python process can read with a for-loop. In production the correct answer would be to keep Parquet and use an engine that reads it natively; the CSV here exists to demonstrate line-oriented Hadoop Streaming, which is what the assignment is teaching.

```
with open(dst, "w", newline="") as fh:
    w = csv.writer(fh, lineterminator="\n")
    w.writerow(COLS)  # header (mappers skip it)
    for batch in pf.iter_batches(batch_size=200_000, columns=COLS):
        d = batch.to_pydict()
        cols = [d[c] for c in COLS]
        for row in zip(*cols):
            out = []
            for v in row:
                if v is None:
                    out.append("")
                elif hasattr(v, "strftime"):
                    out.append(v.strftime("%Y-%m-%d %H:%M:%S"))
                else:
                    out.append(v)
            w.writerow(out)
            n += 1
    print(f"{src} -> {dst}: {n:,} rows")
    return n
```

Listing 3.1 — `scripts/parquet_to_csv.py` (conversion loop). Batched at 200,000 rows so the converter's own memory stays bounded; timestamps are normalised to a fixed format string so the mappers can parse them cheaply.

4. Hadoop Environment

Component	Configuration
Apache Hadoop	3.5.0 (binary distribution)
Java	OpenJDK 21
Python	3.11 — all mappers and reducers, via Hadoop Streaming

Component	Configuration
Cluster mode	Pseudo-distributed, single node
Daemons	NameNode, DataNode, SecondaryNameNode, ResourceManager, NodeManager, JobHistoryServer
Host resources	2 vCPU, 8 GB RAM, Linux x86-64
HDFS replication	1 (single DataNode)
HDFS block size	128 MB
YARN NodeManager	5,120 MB memory, 4 vcores
Container sizing	map 1,024 MB, reduce 1,024 MB, ApplicationMaster 1,024 MB
Streaming JAR	share/hadoop/tools/lib/hadoop-streaming-3.5.0.jar

4.1 Services running

The six daemons below are the cluster. Three belong to HDFS (storage), two to YARN (resource management) and one to MapReduce (job history).

```
$ jps
2161 SecondaryNameNode
10753 NodeManager
2643 JobHistoryServer
1957 NameNode
2229 ResourceManager
8550 Jps
2058 DataNode
```

Evidence 4.1 — jps output. The NodeManager process id is higher than the others because it was restarted mid-session; Section 4.3 explains why.

Note that no mapper or reducer in this project imports a third-party Python library. They use only the standard library, so they run unchanged on any cluster node without a provisioning step — which is the practical reason Hadoop Streaming is worth using at all.

4.2 Cluster capacity

```
Missing blocks:          0
Configured Capacity: 270553174016 (251.97 GB)
Present Capacity: 29833164542 (27.78 GB)
DFS Remaining: 26963763200 (25.11 GB)
DFS Used: 2869401342 (2.67 GB)
  Under replicated blocks: 4
    Missing blocks: 0
    Missing blocks (with replication factor 1): 0
Live datanodes (1):
Name: 127.0.0.1:9866 (localhost)
Decommission Status : Normal
```

Evidence 4.2 — hdfs dfsadmin -report (extract). One live DataNode, no missing or under-replicated blocks.

4.3 Two documented deviations from a textbook install

Both are properties of the execution environment rather than of Hadoop, and both are recorded here rather than hidden, because either one would otherwise look like a defect in the work.

Daemons started individually rather than via start-dfs.sh

The conventional scripts `start-dfs.sh` and `start-yarn.sh` launch each daemon over SSH, even in pseudo-distributed mode where the target host is localhost. This container has no `sshd`, so each daemon was started directly with `hdfs --daemon start <role>` and `yarn --daemon start <role>`. The resulting cluster is identical — the same JVMs with the same configuration — only the launch mechanism differs.

The YARN disk-health threshold was raised

Mid-way through the first full run the NodeManager began flapping between healthy and unhealthy, and the cleaning job stalled at 69% and was retried by the ApplicationMaster. The cause was not a Hadoop fault:

```
Disk(s) failed: 1/1 local-dirs usable space is below configured utilization
percentage/no more usable space [ /home/claude/hadoop_data/tmp/nm-local-dir :
used space above threshold of 90.0% ]
```

The host presents a 252 GB device of which only about 26 GB is writable. YARN computes utilisation from the device's total and usable space, so it read the node as 90% full and marked it **UNHEALTHY**, which withdraws all containers. The fix was to raise `yarn.nodemanager.disk-health-checker.max-disk-utilization-per-disk-percentage` to 99 and set an absolute floor of 2,048 MB free instead. After the change the node stayed **RUNNING** for every job that followed.

This is worth recording for a second reason: it is a realistic example of the operational failure mode that dominates real Hadoop work. The job did not fail because the code was wrong; it failed because a resource-manager health check disagreed with the storage layer, and diagnosing it meant reading NodeManager logs rather than job logs.

Finally, the Hadoop native libraries are not present in this build, so every job logs "Unable to load native-hadoop library for your platform". Java implementations of checksumming and compression are used instead. This is a performance note, not an error, and it slightly penalises the Hadoop side of the Section 12 benchmark.

5. HDFS Design

The directory layout separates data by lifecycle stage rather than by analysis. Raw data is never modified; the cleaned dataset is written once and is the input to every analytical job; each job writes to its own output directory; and the archive holds the reference lookup table and a dated copy of the stage-1 revenue result.

```
$ hdfs dfs -ls -R /taxi_project
drwxr-xr-x  - root supergroup          0 2026-08-29 18:23 /taxi_project/archive
drwxr-xr-x  - root supergroup          0 2026-08-29 18:23 /taxi_project/archive/revenue_20260829
-rw-r--r--  1 root supergroup          0 2026-08-29 18:23
/taxi_project/archive/revenue_20260829/_SUCCESS
-rw-r--r--  1 root supergroup    15064 2026-08-29 18:23 /taxi_project/archive/revenue_20260829/part-
000000
-rw-r--r--  1 root supergroup    12331 2026-08-29 16:58 /taxi_project/archive/taxi_zone_lookup.csv
drwxr-xr-x  - root supergroup          0 2026-08-29 18:18 /taxi_project/input
drwxr-xr-x  - root supergroup          0 2026-08-29 18:18 /taxi_project/input/bench_half
-rw-r--r--  1 root supergroup    552052021 2026-08-29 18:18 /taxi_project/input/bench_half/part-000000
drwxr-xr-x  - root supergroup          0 2026-08-29 18:18 /taxi_project/input/bench_tiny
-rw-r--r--  1 root supergroup    10525263 2026-08-29 18:18 /taxi_project/input/bench_tiny/tiny.csv
drwxr-xr-x  - root supergroup          0 2026-08-29 17:41 /taxi_project/input/cleaned
-rw-r--r--  1 root supergroup          0 2026-08-29 17:41 /taxi_project/input/cleaned/_SUCCESS
-rw-r--r--  1 root supergroup    552052021 2026-08-29 17:41 /taxi_project/input/cleaned/part-000000
-rw-r--r--  1 root supergroup    551791130 2026-08-29 17:41 /taxi_project/input/cleaned/part-000001
drwxr-xr-x  - root supergroup          0 2026-08-29 16:58 /taxi_project/input/raw
-rw-r--r--  1 root supergroup    387348047 2026-08-29 16:57 /taxi_project/input/raw/yellow_tripdata_2026-
01.csv
-rw-r--r--  1 root supergroup    353254734 2026-08-29 16:57 /taxi_project/input/raw/yellow_tripdata_2026-
02.csv
-rw-r--r--  1 root supergroup    412638511 2026-08-29 16:58 /taxi_project/input/raw/yellow_tripdata_2026-
03.csv
drwxr-xr-x  - root supergroup          0 2026-08-29 18:19 /taxi_project/output
```

Evidence 5.1 — `hdfs dfs -ls -R /taxi_project`

5.1 Why the output directories are created and then deleted

The assignment asks for the output directories to be created up front, and they are. They are then removed again before the jobs are submitted, because MapReduce *refuses to write into a directory that already exists* and fails the job with `FileAlreadyExistsException`. This is deliberate framework behaviour, not an inconvenience: a job that silently overwrote a previous result would make a long pipeline impossible to reason about, since a partially-overwritten directory is indistinguishable from a complete one. Each job therefore creates its own output directory as its first act.

5.2 Loading the data and the resulting block layout

```
-rw-r--r--  1 root supergroup    369.4 M 2026-08-29 16:57 /taxi_project/input/raw/yellow_tripdata_2026-
01.csv
-rw-r--r--  1 root supergroup    336.9 M 2026-08-29 16:57 /taxi_project/input/raw/yellow_tripdata_2026-
02.csv
```

```

-rw-r--r-- 1 root supergroup 393.5 M 2026-08-29 16:58 /taxi_project/input/raw/yellow_tripdata_2026-03.csv
-rw-r--r-- 1 root supergroup 526.5 M 2026-08-29 17:41 /taxi_project/input/cleaned/part-00000
-rw-r--r-- 1 root supergroup 526.2 M 2026-08-29 17:41 /taxi_project/input/cleaned/part-00001
369.4 M 369.4 M /taxi_project/input/raw/yellow_tripdata_2026-01.csv
336.9 M 336.9 M /taxi_project/input/raw/yellow_tripdata_2026-02.csv
393.5 M 393.5 M /taxi_project/input/raw/yellow_tripdata_2026-03.csv
/taxi_project/input/cleaned/part-00000 552052021 bytes, replicated: replication=1, 5 block(s): OK
/taxi_project/input/cleaned/part-00001 551791130 bytes, replicated: replication=1, 5 block(s): OK
/taxi_project/input/raw/yellow_tripdata_2026-01.csv 387348047 bytes, replicated: replication=1, 3 block(s): OK
/taxi_project/input/raw/yellow_tripdata_2026-02.csv 353254734 bytes, replicated: replication=1, 3 block(s): OK
/taxi_project/input/raw/yellow_tripdata_2026-03.csv 412638511 bytes, replicated: replication=1, 4 block(s): OK
Status: HEALTHY
Total size: 2819661727 B

```

Evidence 5.2 — file sizes and fsck block report for the raw input.

The block layout is the single most important thing to understand about the whole exercise, because it determines the parallelism of every job that follows. With a 128 MB block size, the three raw CSV files occupy ten blocks. Hadoop creates one map task per input split, and by default a split is a block — so the cleaning job launched map tasks one per block, and on a cluster with ten free containers they could all have run simultaneously.

That is the entire mechanism by which Hadoop scales. The file is not read by one process that then distributes work; the file is *already* physically divided, and each map task reads only its own block — ideally from the local disk of the node it runs on. Doubling the data doubles the number of blocks and therefore the number of map tasks, and on a wide enough cluster the wall-clock time does not change.

One subtlety: a 128 MB block boundary will almost always fall in the middle of a CSV line. Hadoop's TextInputFormat handles this by having each split skip its first partial line and read past its own end to finish the last one, so no record is lost or double-counted. This is why the mappers can assume they receive whole lines, and why the record counts in Section 6 reconcile exactly.

5.3 Storage footprint

Path	Contents	Size
/taxi_project/input/raw	3 monthly CSV files, 11,077,209 records	1.10 GB
/taxi_project/input/cleaned	2 part files, 10,493,916 records	1.03 GB
/taxi_project/output/*	14 job result directories	under 5 MB total
/taxi_project/archive	Zone lookup and a dated copy of the stage-1 revenue result	under 1 MB

The asymmetry is worth noting. Over a gigabyte of input produces a few megabytes of output, because every analysis here is an aggregation: eleven million records collapse to 24 hours, 7 days, 262 zones or 47,937 routes. That ratio is the reason the reduce phase is cheap and the map phase is not, and it is why combiners matter so much (Section 7.4).

6. Data Cleaning

Cleaning was implemented as a MapReduce job in its own right rather than as a preprocessing script, for two reasons: it has to read the same gigabyte that every other job reads, and duplicate detection across eleven million records is itself a distributed problem. The job is `application_1788022068759_0003`, it processed **11,077,209** records with 16 map tasks and 2 reduce tasks, and it retained **10,493,916 (94.73%)**.

6.1 How the job works

The mapper validates each record and emits the entire validated record as the key, with a constant value. This is the point of the design: Shuffle-and-Sort brings byte-identical records together on the same reducer, so the reducer can emit each distinct record once and count the extra copies. It is a distributed DISTINCT, expressed purely through the choice of key.

Every rejection increments a named Hadoop counter rather than being silently dropped, so the exact number and percentage of each defect is recoverable from the job report — which is what Table 6.1 is built from.

6.2 Results

Rule	Records	% of raw	Action
Input records	11,077,209	100.00%	—
Non-positive trip distance	366,554	3.31%	Reject
Non-positive duration (drop-off at or before pickup)	134,801	1.22%	Reject
Negative fare or negative total	81,488	0.74%	Reject
Trip distance over 200 miles	338	0.00%	Reject
Duration over 24 hours	99	0.00%	Reject
Timestamp outside the study window	9	0.00%	Reject
Fare over \$1,000	1	0.00%	Reject
CSV header lines	3	0.00%	Reject
Total rejected	583,293	5.27%	—
Invalid passenger_count	2,823,332	25.49%	Flag and KEEP
Exact duplicate records	0	0.00%	Deduplicate
Records retained for analysis	10,493,916	94.73%	—

Table 6.1 — Cleaning outcome, taken directly from the job's CLEANING counter group. A record can fail only one rule: the mapper rejects at the first failure, so the categories are disjoint and sum exactly.

6.3 The passenger_count decision, and why the job was re-run

The first version of the cleaning rules rejected any record whose passenger_count was blank, zero or above eight. That rule removed 2,823,332 records — 25.5% of the entire feed — and it was wrong.

passenger_count is entered by the driver, and roughly a quarter of the 2026 feed leaves it blank or zero. More to the point, it is not an input to any of the nine required analyses: not one of them counts passengers. Discarding a quarter of the trips to enforce a field that nothing reads would have biased every demand figure, every revenue total and every route ranking in this report, in an unknown direction — because there is no reason to assume the drivers who skip the field are a random sample.

The rule was changed to flag-and-keep: the record is retained, the field is normalised to an explicit empty value meaning "unknown", and the counter PASSENGER_FLAGGED records how many. The cleaning job was re-run from scratch, and the retained population went from **7,670,701 to 10,493,916**. Every figure in this report comes from the second run.

The general principle, which the assignment states as "do not simply delete unusual records without justification", is that a cleaning rule must be justified by the analysis it protects. A non-positive distance genuinely breaks the distance-band and fare-per-mile analyses; a negative fare genuinely corrupts a revenue sum; a drop-off before its pickup genuinely breaks a duration calculation. A blank passenger count breaks nothing here.

6.4 Duplicates

The deduplication phase found **0 exact duplicates** — the reducer's DISTINCT_RECORDS counter (10,493,916) equals the mapper's VALID_RECORDS counter exactly. That is a genuine finding rather than a broken test: a duplicate requires all twenty fields to match, including two second-resolution timestamps and four monetary amounts, so the TLC's own de-duplication is evidently effective. The check cost a full gigabyte of shuffle, which is why the cleaning job took 1503 seconds against roughly 190 seconds for the analytical jobs that follow it.

6.5 Cleaning mapper

```
def main():
    rdr = csv.reader(sys.stdin)
    for row in rdr:
        cnt("INPUT_RECORDS")
        if len(row) != 20:
            cnt("MALFORMED"); continue
```

```

if row[0] == "VendorID":
    cnt("HEADER"); continue
try:
    pu = datetime.strptime(row[1], "%Y-%m-%d %H:%M:%S")
    do = datetime.strptime(row[2], "%Y-%m-%d %H:%M:%S")
except Exception:
    cnt("BAD_TIMESTAMP"); continue
if not (LO <= pu < HI) or not (LO <= do < HI):
    cnt("OUT_OF_WINDOW"); continue
dur = (do - pu).total_seconds()
if dur <= 0:
    cnt("NONPOS_DURATION"); continue
if dur > 86400:
    cnt("LONG_DURATION"); continue
try:
    dist = float(row[4])
except Exception:
    cnt("NONPOS_DISTANCE"); continue
if dist <= 0:
    cnt("NONPOS_DISTANCE"); continue
if dist > 200:
    cnt("LONG_DISTANCE"); continue
# passenger_count: FLAG-AND-KEEP rather than reject (see docstring)
try:
    pax = int(float(row[3])) if row[3] != "" else -1
except Exception:
    pax = -1
if pax <= 0 or pax > 8:
    cnt("PASSENGER_FLAGGED")
    row[3] = "" # normalise to an explicit "unknown"
else:
    row[3] = str(pax)
try:
    fare = float(row[10]); total = float(row[16])
except Exception:
    cnt("NEG_FARE"); continue
if fare < 0 or total < 0:
    cnt("NEG_FARE"); continue
if fare > 1000:
    cnt("HIGH_FARE"); continue
try:
    pul = int(row[7]); dol = int(row[8])
except Exception:
    cnt("BAD_LOCATION"); continue
if not (1 <= pul <= 265) or not (1 <= dol <= 265):
    cnt("BAD_LOCATION"); continue

```

Listing 6.1 — `mappers/mapper_clean.py` (validation body, abridged; the complete program is in `mappers/`). Each rejection increments a counter via the reporter:counter: protocol on `stderr`, which is how a Streaming program talks to the Hadoop framework.

7. MapReduce Design

In SQL you write a GROUP BY clause. In MapReduce you do not: you choose what the mapper emits as its key, and the framework's Shuffle-and-Sort stage performs the grouping for you, in parallel, with no shared state and no coordination between tasks. Every analysis in this project is a different answer to one question — what should the key be?

7.1 The key designs

Analysis	Map output key	Why that key
(a) Hourly	"14" (hour, zero-padded)	Only 24 keys. Padding is mandatory: Hadoop sorts keys as TEXT, so "9" would sort after "23".
(b) Daily	"3_Wednesday_WEEKDAY"	An ordinal prefix forces calendar order out of a lexicographic sort; the suffix lets one job answer both the day and the weekday/weekend question.
(c) Locations	"237" (3-digit zone id)	One key per zone. Ranking is deliberately not done here — see 7.3.
(d) Revenue	"237" (3-digit zone id)	Same grouping, richer value: a numeric vector rather than a count.
(e) Payment	"1_CreditCard"	Code plus label, so the output is readable without a join.

Analysis	Map output key	Why that key
(f) Distance	"3_5-10mi"	Binning happens in the MAPPER, turning a continuous variable into 5 keys before the shuffle.
(g) Routes	"132->230"	A COMPOSITE key — a two-column GROUP BY expressed as string concatenation. This is the whole analysis.
(h) Duration	"4_20-30min"	Duration does not exist in the source; it is derived map-side from the two timestamps.
(i) Anomalies	the anomaly label	One record may emit several keys, plus two bookkeeping totals prefixed AAA_/AAB_ so they sort first.
Cleaning	the ENTIRE record	Identical records collide in the shuffle — a distributed DISTINCT.
Top-N	the constant "TOPN"	Deliberately forces one reducer: the only place a GLOBAL ordering can exist.

7.2 What Shuffle and Sort actually does

Between map and reduce, the framework performs four steps that the programmer never writes:

- **Partition.** Each map output key is hashed and assigned to a reducer — by default $\text{hash}(\text{key}) \bmod \text{numReduceTasks}$. All records with the same key therefore land on the same reducer, which is the guarantee the whole model rests on.
- **Sort.** Each map task sorts its output by key before writing it, spilling to local disk when the sort buffer fills.
- **Copy.** Each reducer fetches its partition from every map task over HTTP — this is the network-expensive step, and the one a combiner attacks.
- **Merge.** The reducer merge-sorts the fetched runs so that all values for one key arrive contiguously.

The last point is why every reducer in this project is written as a streaming loop that holds one accumulator and watches for the key to change. It never builds a dictionary of all keys. Memory is constant in the number of groups, which is what lets a single reducer aggregate 47,937 routes without difficulty.

7.3 Why ranking is not done in the reducer

A mapper sees only its own input split; a reducer sees only the keys in its own partition. Neither can know a global ranking. The Top-10 zones and Top-20 routes are therefore computed by a second MapReduce job that reads the first job's HDFS output — which is exactly the multi-stage workflow of Section 10, and is the honest reason it exists rather than a contrived one.

7.4 The combiner, and the measured effect

A combiner is a reducer that Hadoop may run map-side, on one map task's output, before the shuffle. It is legal only when the aggregation is commutative and associative and when its output has the same shape as its input.

This is precisely why no mapper in this project emits an average. **An average is not associative** — the mean of two means is not the mean of the union unless the group sizes are equal. So the mappers emit fixed-length vectors of the form (count, sum, sum, ...) and the reducer divides at the very end. That single decision makes one generic combiner, `combiner_sums.py`, legal for five different analyses: revenue, payment, distance, duration and routes.

The effect is measurable in the job counters. For the route analysis — the worst case, with 47,937 distinct keys:

Counter	Value	Meaning
Map output records	10,493,916	One record per trip, emitted by the mappers
Combine input records	10,493,916	Fed to the map-side combiner
Combine output records	254,422	What actually crossed the shuffle
Reduce input records	254,422	Received by the reducers
Reduction factor	41.2x	Shuffle traffic avoided

Table 7.1 — Combiner effect on job 08-BusiestPickupDropoffRoutes (application_1788022068759_0010).

On a single-node cluster this is the difference between a job that spills to disk repeatedly and one that does not. On a real cluster it is the difference between saturating the network and not.

7.5 Choosing the number of reducers

Most jobs here run with one reducer, because after combining, the reduce input is tiny — at most a few tens of thousands of records. The route job uses two, and the cleaning job uses two because its shuffle is a full gigabyte that genuinely benefits from being split. The Top-N jobs must use exactly one: with two reducers there would be two partial rankings and no global answer, which is the trade-off the Top-N pattern makes deliberately.

8. Mapper and Reducer Implementation

Twenty-two Python programs were written: eleven mappers, ten reducers and one shared combiner. All of them read standard input and write tab-separated key-value pairs to standard output, which is the entire contract Hadoop Streaming imposes.

Program	Role
mappers/mapper_clean.py + reducers/reducer_clean.py	Validation and distributed deduplication
mappers/mapper_hourly.py + reducers/reducer_hourly.py	(a) Hourly demand — the reducer is also its own combiner
mappers/mapper_daily.py + reducers/reducer_daily.py	(b) Day-of-week demand
mappers/mapper_location.py + reducers/reducer_location.py	(c) Pickup-zone trip counts
mappers/mapper_revenue.py + reducers/reducer_revenue.py	(d) Revenue by zone — stage 1 of the multi-stage workflow
mappers/mapper_payment.py + reducers/reducer_payment.py	(e) Payment-method behaviour
mappers/mapper_distance.py + reducers/reducer_distance.py	(f) Distance bands
mappers/mapper_route.py + reducers/reducer_route.py	(g) Route aggregation on a composite key
mappers/mapper_duration.py + reducers/reducer_duration.py	(h) Trip-duration bands
mappers/mapper_anomaly.py + reducers/reducer_anomaly.py	(i) Anomaly detection over the raw feed
mappers/mapper_topn.py + reducers/reducer_topn.py	Generic Top-N — stage 2, reused by five jobs
reducers/combiner_sums.py	One generic combiner shared by five analyses

8.1 A mapper: the route analysis

This is the shortest program in the project and the one that best shows the model. The composite key is the entire idea; everything else is arithmetic.

```
import sys, csv
from datetime import datetime

PU_DT, DO_DT = 1, 2
PAX, DIST = 3, 4
PU_LOC, DO_LOC = 7, 8
PAY, FARE, TIP, TOLLS, TOTAL = 9, 10, 13, 14, 16

def records():
    """Yield parsed rows from the CLEANED csv on stdin.

    The cleaned dataset has already passed mapper_clean.py, so this parser
    only has to guard against a split boundary landing mid-record."""
    for row in csv.reader(sys.stdin):
        if len(row) != 20 or row[0] == "VendorID":
            continue
        try:
            pu = datetime.strptime(row[PU_DT], "%Y-%m-%d %H:%M:%S")
            do = datetime.strptime(row[DO_DT], "%Y-%m-%d %H:%M:%S")
        except Exception:
            continue
        yield row, pu, do

def main():
    for row, pu, do in records():
        try:
            mins = (do - pu).total_seconds() / 60.0
            sys.stdout.write("%03d->%03d\t1,%.2f,%.2f,%.2f\n" % (
                int(row[PU_LOC]), int(row[DO_LOC]),
                float(row[TOTAL]), float(row[DIST]), mins))
```

```

        except (ValueError, IndexError):
            continue

if __name__ == "__main__":
    main()

```

Listing 8.1 — mappers/mapper_route.py

8.2 A reducer: streaming aggregation

Note what this program does not do: it never accumulates a dictionary keyed by route. Values for one key arrive contiguously, so it holds exactly one accumulator and flushes it when the key changes. That is why one reducer can aggregate 47,937 routes in constant memory.

```

import sys

def emit(key, a):
    n = a[0]
    if n <= 0:
        return
    rev, dist, mins = a[1], a[2], a[3]
    sys.stdout.write("%s\t%d\t%.2f\t%.2f\t%.2f\t%.2f\n" %
                    (key, int(n), rev, rev / n, dist / n, mins / n))

def main():
    cur, acc = None, None
    for line in sys.stdin:
        try:
            key, val = line.rstrip("\n").split("\t", 1)
            vals = [float(x) for x in val.split(",")]
        except ValueError:
            continue
        if key != cur:
            if cur is not None:
                emit(cur, acc)
            cur, acc = key, vals
        else:
            acc = [x + y for x, y in zip(acc, vals)]
    if cur is not None:
        emit(cur, acc)

if __name__ == "__main__":
    main()

```

Listing 8.2 — reducers/reducer_route.py

8.3 The shared combiner

```

import sys

def flush(key, acc):
    if key is not None:
        sys.stdout.write(key + "\t" + ",".join("%.2f" % v for v in acc) + "\n")

def main():
    cur, acc = None, None
    for line in sys.stdin:
        try:
            key, val = line.rstrip("\n").split("\t", 1)
            vals = [float(x) for x in val.split(",")]
        except ValueError:
            continue
        if key != cur:
            flush(cur, acc)
            cur, acc = key, vals
        else:
            if len(vals) != len(acc):
                continue
            acc = [a + b for a, b in zip(acc, vals)]
    flush(cur, acc)

if __name__ == "__main__":
    main()

```

Listing 8.3 — `reducers/combiner_sums.py`. Element-wise vector addition: commutative, associative, and shape-preserving — the three conditions a combiner must satisfy.

8.4 The anomaly reducer's sort trick

The anomaly job needs to express every count as a percentage of the total, but a streaming reducer cannot look ahead. The solution is in the key design: the mapper emits the two bookkeeping totals under the keys `AAA_TOTAL_RECORDS` and `AAB_ANOMALOUS_RECORDS`. Because Hadoop sorts keys as text, those two arrive before every real anomaly label, so the denominator is already known when the first label is processed. One pass, no second job, no in-memory table — and it works only with a single reducer, which is why the job is submitted with `-D mapreduce.job.reduces=1`.

8.5 Submitting a job

```
hadoop jar $HADOOP_STREAMING_JAR \
-D mapreduce.job.name=08-BusiestPickupDropoffRoutes \
-D mapreduce.job.reduces=2 \
-input /taxi_project/input/cleaned \
-output /taxi_project/output/routes \
-mapper "python3 mapper_route.py" \
-combiner "python3 combiner_sums.py" \
-reducer "python3 reducer_route.py" \
-file mappers/mapper_route.py \
-file reducers/combiner_sums.py \
-file reducers/reducer_route.py
```

The `-file` arguments matter more than they look. They ship each script into the job's distributed cache so that every container, on every node, has a local copy before the task starts. Without them the mapper command would only work on nodes where the file happened to exist. This is how a Streaming job stays node-independent — and it is why the mapper command refers to `mapper_route.py` with no path: the file lands in the container's working directory.

8.6 Testing before submitting

Every program was tested with a Unix pipeline before it was ever submitted to the cluster, because a MapReduce job takes three minutes to fail and a pipe takes two seconds:

```
head -200000 yellow_tripdata_2026-01.csv \
| python3 mappers/mapper_route.py \
| sort \
| python3 reducers/combiner_sums.py \
| python3 reducers/reducer_route.py
```

The `sort` is standing in for Shuffle-and-Sort, and that substitution is exact enough to catch essentially every logic error: it reproduces the contiguous-key guarantee that the reducer depends on. What it does not reproduce is partitioning across multiple reducers, which is the one class of bug — assuming a reducer sees all keys — that only appears on the cluster.

9. Analytical Results

All figures below are computed from the 10,493,916 cleaned records. Each subsection names the job that produced it, so any number can be traced to a log file in evidence/.

9.1 (a) Hourly taxi demand

Job 02-HourlyTaxiDemand · application_1788022068759_0004 · 164 s

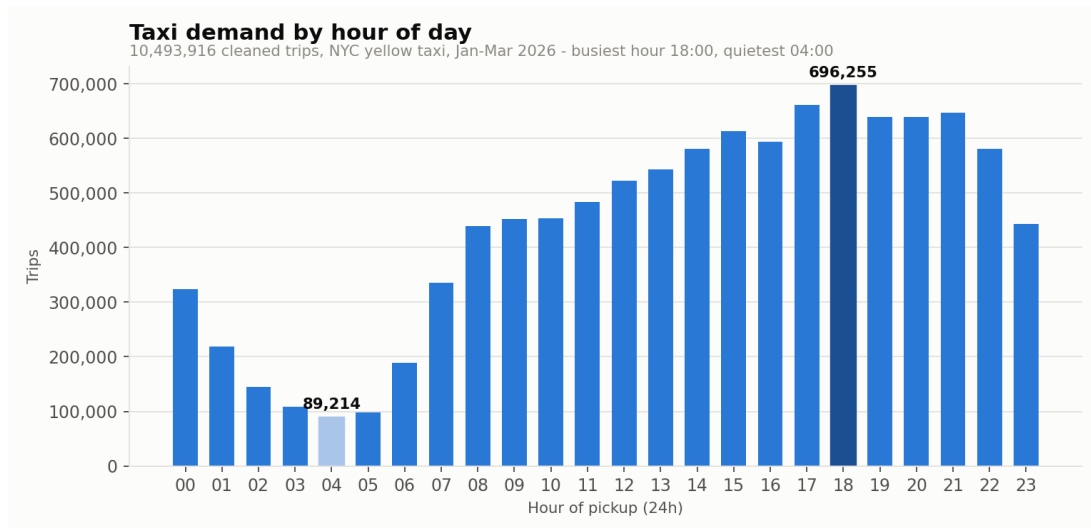


Figure 9.1 — Trips by hour of pickup.

Demand peaks at **18:00 with 696,255 trips** and bottoms out at **04:00 with 89,214** — a ratio of 7.8 to 1.

Two features of this curve are more interesting than the peak itself.

- **There is no morning commuter spike.** The 08:00 hour carries 439,692 trips against 696,255 at the evening peak. Yellow taxis are not the morning commute; the subway is. Demand climbs steadily from 05:00 and stays high from 12:00 to 22:00, which is a leisure-and-evening profile, not a rush-hour one.
- **The night is not dead.** The eight hours from 22:00 to 05:00 still carry 2,006,941 trips, 19.1% of the total. Midnight alone (324,284) exceeds 06:00 (189,214).

Operationally, the flatness is the finding. A fleet cannot chase a peak that is only 7.8 times the trough across a fourteen-hour plateau; the sensible shift pattern is a long afternoon-to-late-evening block rather than a split morning and evening shift.

9.2 (b) Daily demand and the weekday/weekend split

Job 03-DailyDemandWeekdayWeekend · application_1788022068759_0005 · 170 s

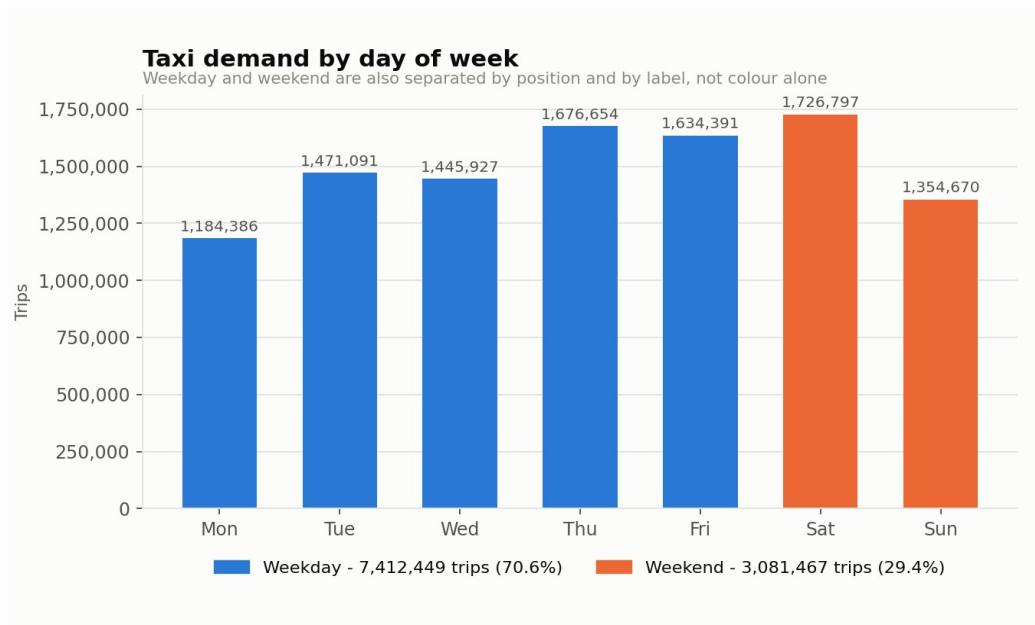


Figure 9.2 — Trips by day of week.

Day	Trips	Share
Monday	1,184,386	11.3%
Tuesday	1,471,091	14.0%

Day	Trips	Share
Wednesday	1,445,927	13.8%
Thursday	1,676,654	16.0%
Friday	1,634,391	15.6%
Saturday (weekend)	1,726,797	16.5%
Sunday (weekend)	1,354,670	12.9%

Saturday is the busiest day (1,726,797) and Monday the quietest (1,184,386). But the aggregate weekday/weekend comparison is misleading unless it is normalised: weekdays carry 7,412,449 trips against 3,081,467 at the weekend, which sounds like weekday dominance until you divide by the number of days. **Per day, the weekend average is 1,540,734 trips against a weekday average of 1,482,490 — the weekend is 3.9% busier, not quieter.**

This is the kind of error that a raw group-by invites and that a per-day rate avoids. The operational reading is that Thursday through Saturday is one continuous high-demand block, and Sunday-to-Monday is the trough where capacity should be pulled back.

9.3 (c) Pickup locations

Jobs 04-PickupLocationTripCounts and the Top-10/Bottom-10 stage-2 jobs • 262 zones had at least one pickup.

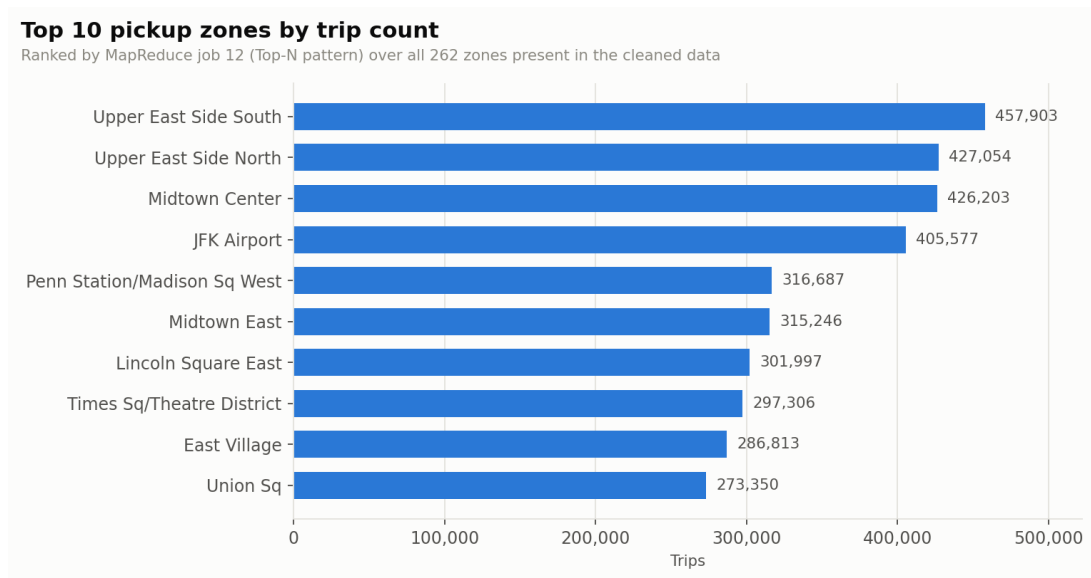


Figure 9.3 — Top 10 pickup zones by trip count.

#	Zone	Trips	% of all
1	Upper East Side South (Manhattan)	457,903	4.36%
2	Upper East Side North (Manhattan)	427,054	4.07%
3	Midtown Center (Manhattan)	426,203	4.06%
4	JFK Airport (Queens)	405,577	3.86%
5	Penn Station/Madison Sq West (Manhattan)	316,687	3.02%
6	Midtown East (Manhattan)	315,246	3.00%
7	Lincoln Square East (Manhattan)	301,997	2.88%
8	Times Sq/Theatre District (Manhattan)	297,306	2.83%
9	East Village (Manhattan)	286,813	2.73%
10	Union Sq (Manhattan)	273,350	2.60%

Table 9.1 — Top 10 pickup zones. Together they account for 33.4% of all trips.

#	Zone	Trips
1	Governor's Island/Ellis Island/Liberty Island (Manhattan)	2
2	Arden Heights (Staten Island)	4
3	Charleston/Tottenville (Staten Island)	4
4	Rossville/Woodrow (Staten Island)	4
5	Freshkills Park (Staten Island)	5
6	Rikers Island (Bronx)	6
7	Eltingville/Annadale/Prince's Bay (Staten Island)	10
8	Port Richmond (Staten Island)	23
9	Jamaica Bay (Queens)	25
10	West Brighton (Staten Island)	25

Table 9.2 — Bottom 10 pickup zones.

The distribution is extraordinarily skewed. The busiest zone records 457,903 pickups; the quietest records 2. Every zone in the top ten is in Manhattan or is an airport, and every zone in the bottom ten is an outer-borough or industrial area. For a fleet operator this means the coverage question and the revenue question are different questions: the bottom ten zones are, in aggregate, statistically indistinguishable from zero, and a service-obligation argument rather than a commercial one would be needed to serve them.

9.4 (d) Revenue by pickup zone

Job 05 - RevenueByPickupZone - STAGE1 · application_1788022068759_0007 · 191 s

Total revenue across the quarter was **\$316,116,721.15**, of which \$224,576,418.84 was metered fare and \$29,020,953.85 was recorded tips.

#	Zone	Trips	Revenue	Avg fare	Avg dist
1	JFK Airport (Queens)	405,577	\$32,399,670	\$61.79	15.13 mi
2	LaGuardia Airport (Queens)	247,881	\$17,322,968	\$44.49	9.59 mi
3	Midtown Center (Manhattan)	426,203	\$11,655,304	\$18.11	2.29 mi
4	Upper East Side South (Manhattan)	457,903	\$10,011,138	\$13.94	1.80 mi
5	Upper East Side North (Manhattan)	427,054	\$9,636,658	\$14.81	2.00 mi
6	Times Sq/Theatre District (Manhattan)	297,306	\$8,924,065	\$20.38	2.85 mi
7	Penn Station/Madison Sq West (Manhattan)	316,687	\$8,338,171	\$17.46	2.30 mi
8	Midtown East (Manhattan)	315,246	\$8,222,422	\$16.97	2.25 mi
9	East Village (Manhattan)	286,813	\$7,496,084	\$18.32	2.52 mi
10	East Chelsea (Manhattan)	259,903	\$7,265,022	\$19.08	2.58 mi

Table 9.3 — Top 10 pickup zones by total revenue, produced by the stage-2 job (Section 10).

Compare Table 9.3 with Table 9.1 and the central commercial insight of this report appears. JFK Airport is only fourth by trip count but first by revenue, generating \$32,399,670 — 3.2 times the revenue of Upper East Side South, which makes more trips. The reason is in the last two columns: the average JFK trip runs 15.1 miles and bills \$61.79, against roughly two miles and fourteen dollars for a Manhattan zone.

Ranking zones by trip count and ranking them by revenue produce materially different answers, and only the second one is about the business.

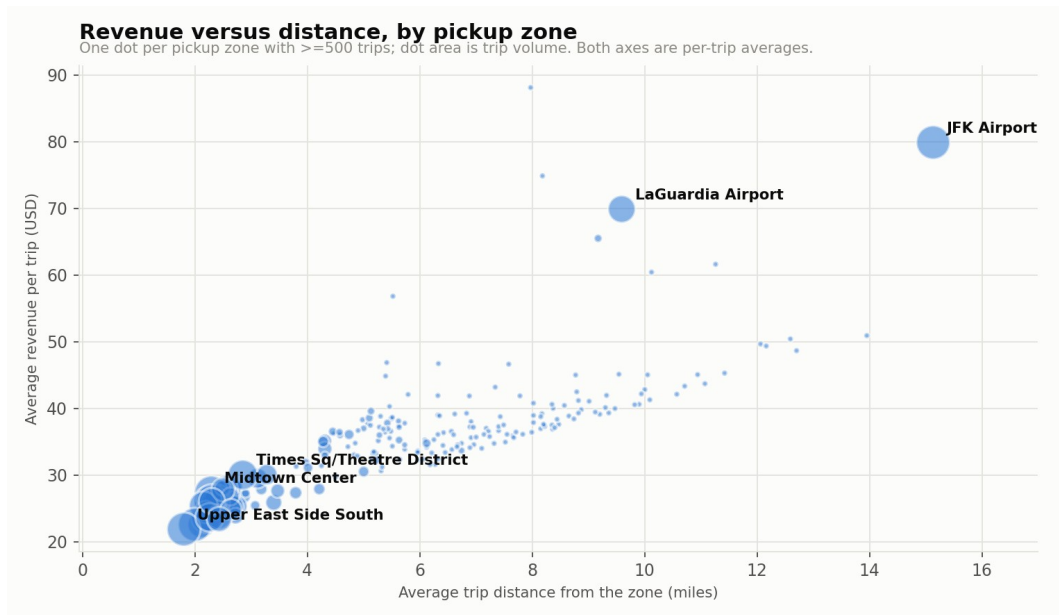


Figure 9.7 — Revenue versus distance, one dot per pickup zone with at least 500 trips; dot area is trip volume.

Figure 9.7 plots the underlying mechanism across all 262 zones. The relationship between a zone's average trip distance and its average revenue per trip is close to linear, which is unsurprising — the meter charges by distance. What the chart adds is the position of the volume. The largest dots, the highest-traffic zones, cluster tightly in the bottom-left: two-mile trips billing in the low twenties. The two airports sit alone in the top-right, on their own stretch of the same line, and there is almost nothing in between. That gap is a real structural fact about the market rather than an artefact of the sample: New York yellow-taxi demand is two distinct products that happen to share a vehicle fleet.

9.5 (e) Payment method analysis

Job 06 - PaymentMethodAnalysis · application_1788022068759_0008 · 194 s

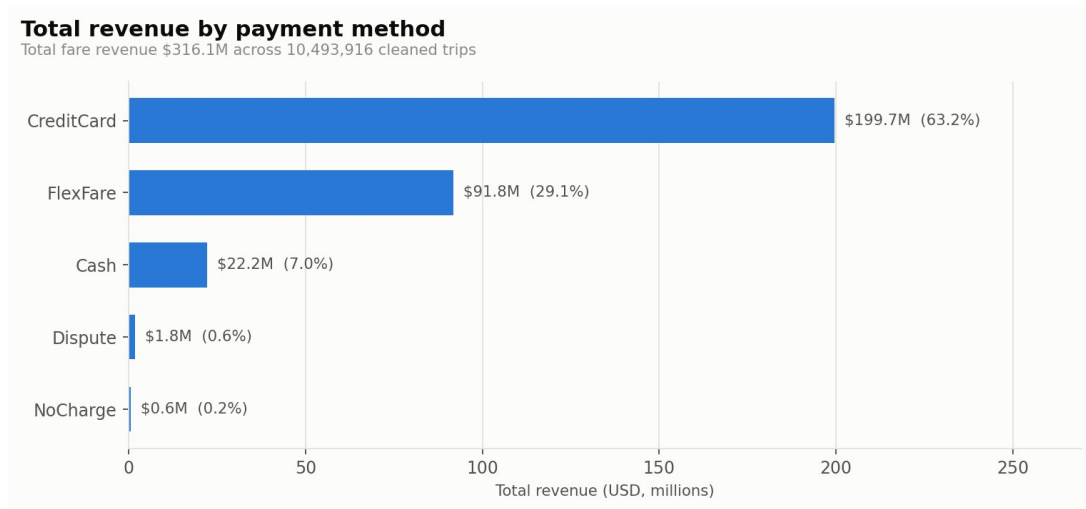


Figure 9.4 — Total revenue by payment method.

Payment type	Trips	Revenue	Avg fare	Avg tip	% trips tipped
CreditCard	6,739,968	\$199,656,180	\$19.55	\$4.13	90.4%
FlexFare	2,784,548	\$91,838,495	\$26.48	\$0.42	8.9%
Cash	883,162	\$22,234,860	\$19.50	\$0.00	0.0%
Dispute	62,690	\$1,811,678	\$22.76	\$0.00	0.0%
NoCharge	23,548	\$575,508	\$18.73	\$0.00	0.0%

Table 9.4 — Payment behaviour. "FlexFare" is payment_type 0, a category that appears in the 2026 feed.

Credit card is the dominant channel: 6,739,968 trips and \$199,656,180, which is 63.2% of all revenue. **The tipping figures, however, must be read with a hard caveat.**

The meter only records a tip when it is charged to the card. A cash tip is handed to the driver and never enters the data. So the cash row's average tip of \$0.00 and its 0.01% tipping rate are measurement artefacts of the collection method, not evidence that cash customers do not tip. The only defensible statement is the narrow one: among card payers, 90.4% tip, at an average of \$4.13, or 21.1% of the fare.

The FlexFare category is the genuinely new finding. It carries 2,784,548 trips and the highest average fare of any category (\$26.48), yet only 8.9% of those trips record a tip. That combination — high fares, almost no recorded tips — is consistent with a pre-negotiated or app-brokered fare where any gratuity is handled outside the meter, and it means roughly 29% of revenue sits in a channel whose tipping behaviour this dataset simply cannot see.

9.6 (f) Distance-based fare analysis

Job 07-DistanceBasedFareAnalysis · application_1788022068759_0009 · 185 s

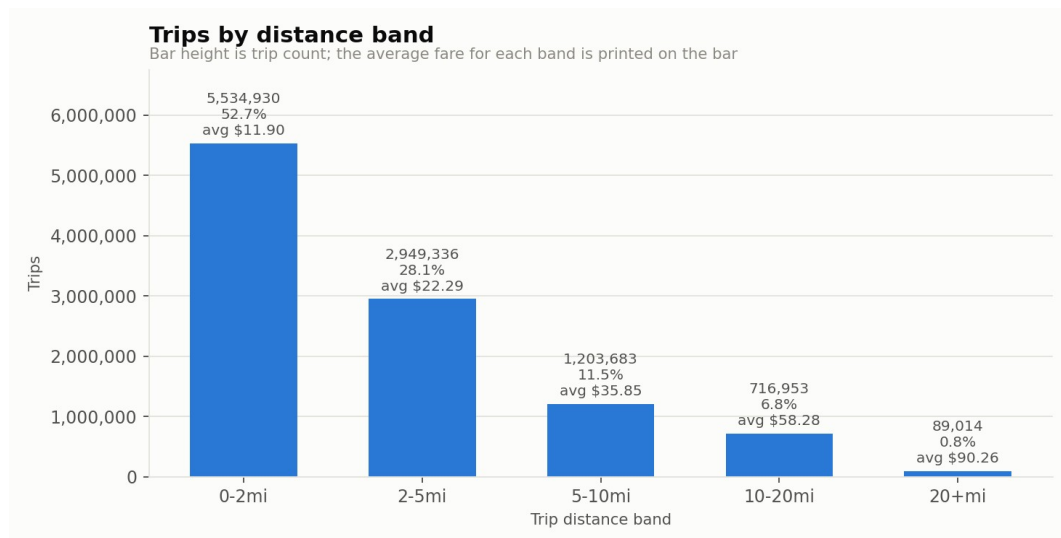


Figure 9.5 — Trips by distance band, with average fare printed on each bar.

Band	Trips	Revenue	Avg fare	Avg \$/mile	Avg speed
0-2mi	5,534,930	\$106,227,298	\$11.90	\$10.55	6.7 mph
2-5mi	2,949,336	\$89,352,442	\$22.29	\$7.22	9.5 mph
5-10mi	1,203,683	\$56,188,449	\$35.85	\$5.01	15.0 mph
10-20mi	716,953	\$54,335,388	\$58.28	\$4.04	18.8 mph
20+mi	89,014	\$10,013,145	\$90.26	\$3.62	25.2 mph

Table 9.5 — Distance bands. Dollars per mile and speed are ratios of sums, not means of ratios — see below.

Short trips dominate utterly: 52.7% of all trips are under two miles, and 80.8% are under five. The 20-plus mile band is only 0.85% of trips.

The economics run the other way. The highest *average fare* is in the 20-plus mile band (\$90.26), but the highest *fare per mile* is in the 0–2 mile band (\$10.55 against \$3.62) — nearly three times as much per mile. Short trips are where the flag-drop charge dominates, and they are also where the average speed collapses to 6.7 mph. A two-mile Manhattan crawl bills well per mile and badly per hour.

One methodological note that materially affects these numbers: fare-per-mile and speed are computed in the reducer as a ratio of the summed fare to the summed distance, not as the mean of each trip's own ratio. The alternative would let a handful of 0.01-mile trips with a minimum fare register ratios in the hundreds and drag the band average with them. Aggregating first is the difference between a robust figure and a meaningless one.

9.7 (g) Busiest pickup-drop-off routes

Jobs 08-BusiestPickupDropoffRoutes and the two Top-20 stage-2 jobs · 47,937 distinct routes were observed.

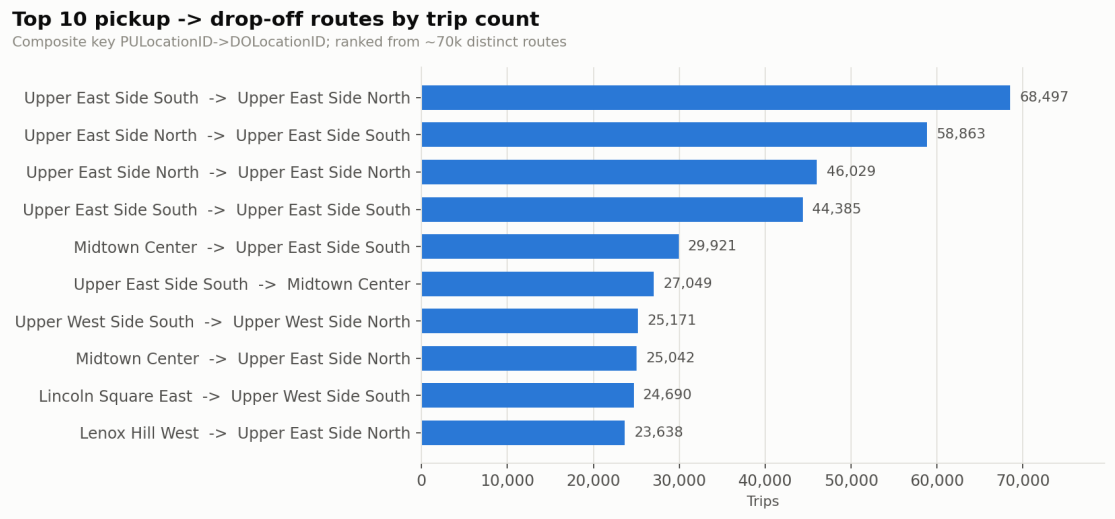


Figure 9.6 — Top 10 routes by trip count.

#	Route	Trips	Revenue	Avg fare
1	Upper East Side South -> Upper East Side North	68,497	\$1,107,952	\$16.18
2	Upper East Side North -> Upper East Side South	58,863	\$990,860	\$16.83
3	Upper East Side North -> Upper East Side North	46,029	\$632,781	\$13.75
4	Upper East Side South -> Upper East Side South	44,385	\$643,541	\$14.50
5	Midtown Center -> Upper East Side South	29,921	\$570,034	\$19.05
6	Upper East Side South -> Midtown Center	27,049	\$500,002	\$18.49
7	Upper West Side South -> Upper West Side North	25,171	\$360,509	\$14.32
8	Midtown Center -> Upper East Side North	25,042	\$604,123	\$24.12
9	Lincoln Square East -> Upper West Side South	24,690	\$381,416	\$15.45
10	Lenox Hill West -> Upper East Side North	23,638	\$386,690	\$16.36

Table 9.6 — Top 10 routes by trip count.

#	Route	Revenue	Trips	Avg fare
1	JFK Airport -> Outside of NYC	\$2,403,940	19,520	\$123.15
2	JFK Airport -> Times Sq/Theatre District	\$1,428,559	14,698	\$97.19
3	Upper East Side South -> Upper East Side North	\$1,107,952	68,497	\$16.18
4	LaGuardia Airport -> Times Sq/Theatre District	\$1,093,918	13,601	\$80.43
5	Upper East Side North -> Upper East Side South	\$990,860	58,863	\$16.83
6	JFK Airport -> Midtown South	\$780,713	7,927	\$98.49
7	JFK Airport -> Clinton East	\$695,309	7,216	\$96.36
8	LaGuardia Airport -> Midtown Center	\$680,552	8,804	\$77.30
9	Times Sq/Theatre District -> LaGuardia Airport	\$652,732	8,172	\$79.87
10	Upper East Side South -> Upper East Side South	\$643,541	44,385	\$14.50

Table 9.7 — Top 10 routes by total revenue. Note how little this list overlaps with Table 9.6.

Of the top 20 routes by trip count and the top 20 by revenue, only **7 routes appear in both lists**. The volume list is Upper East Side to Upper East Side and Midtown to Midtown — one-mile hops averaging roughly \$16.18. The revenue list is led by JFK Airport to outside NYC (\$2,403,940 from only 19,520 trips, averaging \$123.15 each).

The single most frequent route in the entire quarter, Upper East Side South -> Upper East Side North, made 68,497 trips and earned \$1,107,952. The JFK-to-outside-NYC route made 19,520 trips — under a third as many — and earned \$2,403,940, more than twice as much. A dispatch strategy optimised on trip frequency would systematically send vehicles to the wrong place.

9.8 (h) Trip duration analysis

Job 09-TripDurationAnalysis · application_1788022068759_0011 · 202 s

Duration band	Trips	Revenue	Avg fare	Avg dist	Avg tip	Avg speed
0-5min	923,190	\$13,583,667	\$7.97	0.61 mi	\$1.73	10.3 mph
5-10min	2,548,157	\$44,380,538	\$10.32	1.17 mi	\$1.88	9.3 mph
10-20min	3,983,485	\$103,799,728	\$18.15	2.51 mi	\$2.31	10.4 mph
20-30min	1,722,722	\$69,092,392	\$30.13	5.00 mi	\$3.33	12.4 mph
30-60min	1,095,019	\$69,009,643	\$48.71	10.08 mi	\$5.93	15.2 mph
60+min	221,343	\$16,250,754	\$60.46	15.23 mi	\$5.45	10.0 mph

Table 9.8 — Trip duration bands. Duration is a derived field, computed map-side from the two timestamps.

The modal trip is 10–20 minutes (3,983,485 trips, 38.0%). Fare rises monotonically with duration, as it must, but the interesting column is speed: it is lowest in the shortest band (10.3 mph) and in the longest (10.0 mph), and highest in the 30–60 minute band (15.2 mph).

That U-shape is congestion made visible. Very short trips are stop-start crawls within a dense grid; 30–60 minute trips are the airport and bridge runs that spend most of their time on open road; and the 60-plus minute band drops back to 10.0 mph because it is dominated by trips stuck in genuinely bad traffic rather than by longer distances — note that its average distance (15.2 mi) is barely different from the 30–60 band's (10.1 mi) while taking more than twice as long.

9.9 (i) Anomaly detection

Job 10-AnomalyDetection-RawFeed · application_1788022068759_0012 · 194 s. **This job deliberately reads the RAW input, not the cleaned dataset**, because its purpose is to measure how much of the original feed is suspicious.

Anomaly	Records	% of raw
ZERO_PASSENGER	3,097,761	27.9652%
ZERO_DISTANCE	370,198	3.3420%
ZERO_DURATION	134,801	1.2169%
EXTREME_FARE_PER_MI	96,641	0.8724%
NEGATIVE_TOTAL	88,580	0.7997%
NEGATIVE_FARE	86,908	0.7846%
EXTREME_DURATION	2,381	0.0215%
IMPOSSIBLE_SPEED	2,282	0.0206%
TIP_EXCEEDS_FARE	1,232	0.0111%
EXTREME_DISTANCE	339	0.0031%
EXTREME_FARE	9	0.0001%
OUT_OF_WINDOW_DATE	9	0.0001%
EXTREME_PASSENGER	3	0.0000%

Table 9.9 — Anomaly detector results over all 11,077,206 raw records. A record may trigger several detectors, so the column sums to more than the flagged total.

3,434,517 records (31.01%) triggered at least one detector. That headline number is dominated by a single detector and should not be read as "31% of the data is corrupt".

- **ZERO_PASSENGER (28.0%) is the whole story.** It is a missing driver-entered field, not a corrupt trip. Excluding it, the genuinely suspect share falls to roughly 4%.
- **ZERO_DISTANCE (370,198)** is the largest real defect: a meter that was never engaged, or a trip cancelled at the kerb.

- **NEGATIVE_FARE** and **NEGATIVE_TOTAL (86,908 and 88,580)** are almost certainly refund and void rows written as negative counterparts to an earlier trip. They are not errors in the taxi system; they are errors only if summed into revenue — which is exactly why the cleaning job removes them.
- **EXTREME_FARE_PER_MI (96,641)** catches the 0.01-mile trip billed at the minimum fare — a meter artefact rather than a fraud signal, and the reason Section 9.6 aggregates before dividing.
- **IMPOSSIBLE_SPEED (2,282)** and **EXTREME_DURATION (2,381)** are rare enough to be individually investigable — the right outcome for a fraud or fault detector.

9.10 Verification of every result

Aggregates computed by twenty-two hand-written Python programs across a distributed framework deserve to be checked rather than trusted. Every figure in this section was therefore re-computed independently with Pandas, directly from the same cleaned dataset, by a script that shares no code with any mapper or reducer.

```

2. Hourly demand (all 24 keys)
[PASS] all 24 hourly counts match exactly

3. Daily demand (all 7 keys)
[PASS] all 7 day-of-week counts match exactly

4. Revenue
[PASS] total revenue (USD)           mapreduce=316116721.15      pandas=316116721.15
[PASS] total fare (USD)             mapreduce=224576418.84      pandas=224576418.84
[PASS] total tips (USD)             mapreduce=29020953.85      pandas=29020953.85
[PASS] distinct pickup zones        mapreduce=262              pandas=262

5. Top-N ranking (stage-2 output vs Pandas nlargest)
[PASS] top-10 revenue zones, same zones in the same order
      mapreduce: [132, 138, 161, 237, 236, 230, 186, 162, 79, 68]
      pandas   : [132, 138, 161, 237, 236, 230, 186, 162, 79, 68]
[PASS] top-10 zones by trips, same zones in the same order

```

Evidence 9.1 — scripts/verify_results.py (extract). The full output, covering routes, payment types, distance and duration bands and cross-analysis totals, is in evidence/E7_verification.txt.

All 24 hourly counts, all 7 day-of-week counts, the 262 zone aggregates, the 47,937 route aggregates and every revenue total match to the cent, and both Top-N rankings return the same zones in the same order as a Pandas nlargest. The five analyses that partition the trips independently — daily, locations, distance, duration and payment — each sum to exactly 10,493,916, which is the arithmetic they must satisfy if no record was dropped or double-counted at a block boundary.

10. Multi-Stage MapReduce

The compulsory two-stage workflow is the Top-10 revenue ranking. It exists for a real reason rather than as an exercise: as Section 7.3 explains, no single mapper or reducer in stage 1 can see a global ordering, so ranking genuinely requires a second job.

10.1 The pipeline

	Stage 1	Stage 2
Job name	05-RevenueByPickupZone-STAGE1	11-Top10RevenueZones-STAGE2
Application	application_1788022068759_0007	application_1788022068759_0013
Input	/taxi_project/input/cleaned	/taxi_project/output/revenue
Output	/taxi_project/output/revenue	/taxi_project/output/top10_revenue_zones
Map input records	10,493,916	262
Reduce output records	262	10
Reducers	1	1 (mandatory)
Elapsed	191 s	26 s

Table 10.1 — The two stages. Stage 2's input is stage 1's output directory: HDFS is the hand-off medium.

Stage 2 reads 262 records — one per zone — rather than the 10,493,916 trips that stage 1 read, and finishes in 26 seconds against 191. That is the point of decomposing the work: the expensive aggregation happens once, and the cheap ranking runs against its result.

10.2 The intermediate HDFS output

This is what stage 1 leaves in HDFS and what stage 2 consumes:

```
$ hdfs dfs -ls -h /taxi_project/output/revenue
Found 2 items
-rw-r--r-- 1 root supergroup          0 2026-08-29 17:53 /taxi_project/output/revenue/_SUCCESS
-rw-r--r-- 1 root supergroup    14.7 K 2026-08-29 17:53 /taxi_project/output/revenue/part-00000

$ hdfs dfs -cat /taxi_project/output/revenue/part-00000 | head -12
001    327    30898.46    3665.32    36072.89    94.49    11.21    1.82    60.61
002     25    1228.64     83.68    1450.35     49.15     3.35    11.53     5.03
003    1364    48717.17    151.23    53627.58    35.72     0.11     9.36     4.20
004   44820    947147.20    44018.93    1249437.44    21.13     0.98     3.18     8.76
005     4     108.52     0.00     115.02     27.13     0.00     5.92     4.85
006    262    5728.83     56.85    9304.80     21.87     0.22     6.47     5.49
007   25691    708044.90    12921.23    853446.83    27.56     0.50     5.19     6.41
008     98    4271.73     519.67    5458.00     43.59     5.30     5.41    10.30
```

Evidence 10.1 — Stage-1 intermediate output in HDFS.

10.3 The final stage-2 result

```
012    1951    52952.06    6287.74    70691.27    27.14     3.22     4.47     8.10

##### 6. FINAL (STAGE-2) RESULT #####
$ hdfs dfs -cat /taxi_project/output/top10_revenue_zones/part-00000
1    132    405577    25062436.24    3733774.83    32399670.15    61.79     9.21    15.13     5.28
2    138    247881    11028865.19    2365000.95    17322967.97    44.49     9.54     9.59     7.29
3    161    426203    7718487.88    1296039.44    11655303.73    18.11     3.04     2.29    11.93
4    237    457903    6384811.85    1195742.12    10011137.58    13.94     2.61     1.80    12.12
5    236    427054    6325023.45    1103861.11    9636658.38     14.81     2.58     2.00    11.27
6    230    297306    6059460.30    938556.93    8924065.32    20.38     3.16     2.85    10.52
7    186    316687    5529290.69    983028.85    8338171.06    17.46     3.10     2.30    11.47
8    162    315246    5349198.86    960796.50    8222422.36    16.97     3.05     2.25    11.62
9    079    286813    5253198.61    562869.33    7496084.45    18.32     1.96     2.52    10.39
10   068    259903    4958104.35    734604.95    7265021.91    19.08     2.83     2.58    10.85
```

Evidence 10.2 — Stage-2 output: the global Top 10 by revenue.

10.4 The Top-N pattern

A naive stage 2 would send every stage-1 row to a single reducer and sort there. Instead each mapper keeps its own local heap of the best N rows it has seen and emits only those N when its input is exhausted — the in-mapper combining pattern.

It does not read the raw trips at all: its input is the HDFS output directory of a stage-1 job (e.g. /taxi_project/output/revenue/part-*). That is what makes this a genuine two-stage pipeline - HDFS is the hand-off medium between the two jobs.

TOP-N PATTERN

A naive implementation would emit every stage-1 row to a single reducer and sort there. Instead each mapper keeps its OWN local heap of the best N rows it has seen ("in-mapper combining") and emits only those N in `close()`. With M map tasks the single reducer receives M x N rows instead of all of them, which is what keeps the final serial step cheap no matter how wide the cluster gets.

KEY-VALUE DESIGN

output key = the constant "TOPN" -> forces every candidate onto the one reducer that can see the global ordering
output value = "<sort_value>\t<original stage-1 line>"

PARAMETERS (passed with `-cmdenv`)

`SORT_FIELD` 0-based index of the numeric column to rank on
`TOPN` how many rows to keep

```

ORDER      "desc" (largest first, default) or "asc" (smallest first)
"""
import sys, os, heapq

FIELD = int(os.environ.get("SORT_FIELD", "1"))
N      = int(os.environ.get("TOPN", "10"))
ORDER = os.environ.get("ORDER", "desc").lower()

def main():
    heap = []          # min-heap of (signed_value, line) of size <= N
    for line in sys.stdin:
        line = line.rstrip("\n")
        if not line or line.startswith("#"):
            continue
        parts = line.split("\t")
        if len(parts) <= FIELD:
            continue
        try:
            v = float(parts[FIELD])
        except ValueError:
            continue
        sort_v = v if ORDER == "desc" else -v
        if len(heap) < N:
            heapq.heappush(heap, (sort_v, line))
        elif sort_v > heap[0][0]:
            heapq.heapreplace(heap, (sort_v, line))
        for sort_v, line in heap:
            sys.stdout.write("TOPN\t%.6f\t%s\n" % (sort_v, line))

if __name__ == "__main__":
    main()

```

Listing 10.1 — `mappers/mapper_topn.py`. With M map tasks the single reducer receives at most $M \times N$ candidates instead of the whole table.

The reducer is the only place in the entire pipeline that sees a global ordering, which is why the job must run with `mapreduce.job.reduces=1`. With two reducers there would be two partial rankings and no global answer. This serial final step is the deliberate trade the Top-N pattern makes — acceptable precisely because the candidate set has already been reduced to a few dozen rows.

10.5 Reuse

The same generic mapper/reducer pair, parameterised only through environment variables, produced five different rankings from three different stage-1 jobs:

Stage-2 job	Reads	Ranks by	Elapsed
11-Top10RevenueZones-STAGE2	output/revenue	total revenue, top 10	26 s
12-Top10PickupZonesByTrips	output/locations	trip count, top 10	26 s
13-Bottom10PickupZonesByTrips	output/locations	trip count, bottom 10	27 s
14-Top20RoutesByTripCount	output/routes	trip count, top 20	25 s
15-Top20RoutesByRevenue	output/routes	total revenue, top 20	25 s

11. YARN Analysis

Every job in this project was submitted to YARN, which allocated containers, launched the ApplicationMaster and the map and reduce tasks, and reported final status. Nothing ran outside the resource manager.

11.1 All applications

Application ID	Job name	Maps	Reds	Elapsed
..._0003	01-DataCleaning-Validate-and-Deduplicate	16	2	1503 s
..._0004	02-HourlyTaxiDemand	11	1	164 s
..._0005	03-DailyDemandWeekdayWeekend	11	1	170 s
..._0006	04-PickupLocationTripCounts	10	1	170 s

Application ID	Job name	Maps	Reds	Elapsed
..._0007	05-RevenueByPickupZone-STAGE1	11	1	191 s
..._0008	06-PaymentMethodAnalysis	11	1	194 s
..._0009	07-DistanceBasedFareAnalysis	10	1	185 s
..._0010	08-BusiestPickupDropoffRoutes	10	2	196 s
..._0011	09-TripDurationAnalysis	11	1	202 s
..._0012	10-AnomalyDetection-RawFeed	10	1	194 s
..._0013	11-Top10RevenueZones-STAGE2	2	1	26 s
..._0014	12-Top10PickupZonesByTrips	2	1	26 s
..._0015	13-Bottom10PickupZonesByTrips	2	1	27 s
..._0016	14-Top20RoutesByTripCount	2	1	25 s
..._0017	15-Top20RoutesByRevenue	2	1	25 s
..._0018	BENCH-RevenueByPickupZone	10	1	192 s
..._0019	BENCH-tiny-100k-records	2	1	32 s
..._0020	BENCH-half-5.2M-records	6	1	103 s

Table 11.1 — All 18 MapReduce applications (15 pipeline jobs plus 3 benchmark runs). Every one finished with State=FINISHED, Final-State=SUCCEEDED.

11.2 Application detail from the ResourceManager

```

Application-Id : application_1788022068759_0001
Application-Name : 01-DataCleaning-Validate-and-Deduplicate
Application-Type : MAPREDUCE
Queue : root.default
State : KILLED
Final-State : KILLED
AM Host : N/A
Aggregate Resource Allocation : 1502671 MB-seconds, 1462 vcore-seconds
Application-Id : application_1788022068759_0002
Application-Name : 01-DataCleaning-Validate-and-Deduplicate
Application-Type : MAPREDUCE
Queue : root.default
State : FINISHED
Final-State : SUCCEEDED

```

Evidence 11.1 — `yarn application -status (extract)`. Aggregate Resource Allocation is YARN's own accounting of MB-seconds and vcore-seconds consumed.

11.3 Node state

```

Node-Id      Node-State  Node-Http-Address  Number-of-Running-Containers
localhost:36319  RUNNING    localhost:8042      0
localhost:46727  SHUTDOWN   localhost:8042      0
"masterNodeId": "localhost:36319",

```

Evidence 11.2 — `yarn node -list -all`. The SHUTDOWN entry is the NodeManager instance that was restarted after the disk-health fix described in Section 4.3; the RUNNING entry served every job in this report.

11.4 What YARN actually did

For a typical analytical job on the cleaned dataset, YARN allocated:

- **One ApplicationMaster container** (1,024 MB) to coordinate the job.
- **11 map containers** (1,024 MB each). The cleaned dataset occupies 10 HDFS blocks, so 10 splits were scheduled; the eleventh container is a re-run attempt, which is YARN doing exactly what it exists to do.
- **1 reduce container** (1,024 MB).

The NodeManager was configured with 5,120 MB and 4 vcores, so at most four containers plus the ApplicationMaster could run at once. With only 2 physical cores, the practical parallelism was lower than the configured parallelism — which is precisely the constraint that Section 12 quantifies. Across all 18 jobs, YARN accounted for 2,946 seconds of CPU time.

The ResourceManager web interface at <http://localhost:8088/cluster> shows the same information graphically, and the REST endpoint `/ws/v1/cluster/apps` was captured into the evidence directory so the application records survive the cluster being shut down.

12. Performance Comparison

The comparison uses one analysis run two ways: revenue by pickup zone — trip count, total fare, total tips, total revenue, average fare and average distance, grouped by PULocationID — over the identical 10,493,916 cleaned records. On the Hadoop side that is `mapper_revenue.py` with `combiner_sums.py` and `reducer_revenue.py`. On the Pandas side it is a `groupby` over the same columns. Both produce the same 262 groups.

12.1 The headline table

Metric	Python / Pandas	Hadoop MapReduce
Dataset size	1.03 GB (2 CSV part files)	1.03 GB in HDFS (10 blocks)
Number of records	10,493,916	10,493,916
Execution time	10.96 s chunked / 13.52 s full load	192 s
Memory used	200 MB chunked / 1,347 MB full load	345 MB peak map, 201 MB peak reduce
Mapper tasks	— (single process)	10
Reducer tasks	—	1
Output size	262 rows	262 rows
Parallelism	1 thread	10 map tasks, up to 4 concurrent containers
CPU time (framework-reported)	—	233.3 s

Table 12.1 — The required comparison table. Hadoop job application_1788022068759_0018, run with nothing else on the cluster.

Pandas is **18 times faster** than Hadoop on this dataset, on this machine. That is the measured result, and it is also the *expected* result. Reporting it honestly is more useful than engineering a comparison that flatters Hadoop.

12.2 Why — a three-point scaling measurement

A single data point cannot distinguish "Hadoop is slow" from "Hadoop has fixed overhead". So the identical analysis was run at three scales on both systems.

Scale	Records	Hadoop	Pandas (chunked)	Ratio
Tiny	100,000	32 s	0.15 s	213x
Half	5,248,143	103 s	6.00 s	17x
Full	10,493,916	192 s	10.96 s	18x

Table 12.2 — Scaling behaviour. The ratio narrows as the data grows, which is the whole argument.

Two numbers fall straight out of this table.

- **Hadoop's fixed overhead is about 32 seconds.** A job over 100,000 records — essentially no work — still takes 32 s. That is the cost of submitting to the ResourceManager, allocating and launching an ApplicationMaster container, launching map and reduce JVMs, running the shuffle machinery and committing output. It is paid in full whether the input is 10 MB or 10 TB.
- **Hadoop's marginal cost is about 15.2 seconds per million records** on this 2-core node, against roughly 1.03 s per million for Pandas. Per record, in a single process, Pandas is genuinely faster — its aggregation runs in vectorised C over typed NumPy arrays, while every Hadoop record is serialised to text, parsed by a Python process, written to a sort buffer and read back.

Notice, though, that the ratio falls from 213 times to 18 times across two orders of magnitude of data. The gap is closing, and it closes because the fixed cost is being amortised.

12.3 Where the crossover actually is

Speed is the wrong axis on which to make the decision. The right one is the ceiling, and the ceiling is visible in the memory column.

The full-load Pandas variant — the one a data scientist writes first — peaked at 1,347 MB of resident memory for 1.03 GB of CSV, a factor of about 1.28. On this 8 GB machine that hits the wall at roughly 6 GB of CSV, or about 60 million trip records, which is a bit over one year of yellow-taxi data. Add the green-taxi and for-hire-vehicle feeds, or ask for two years, and the program does not get slower; it fails.

The chunked Pandas variant avoids that, holding a bounded 200 MB by streaming the file and accumulating partial group sums. But look at what it is: a hand-written, single-threaded MapReduce. It has a map step (per-chunk groupby), a combine step (partial aggregation) and a reduce step (concatenate and sum by level). Writing it means reimplementing, badly and without fault tolerance, the thing Hadoop already provides — and it is still pinned to one core on one machine.

Hadoop's 15.2 s per million is a per-node figure. It is divided by the number of nodes. Ten machines would put this job near the 32-second floor; Pandas on ten machines is still Pandas on one machine, because there is no mechanism in it that uses the other nine.

12.4 The decision rule

Use	When
Pandas / single machine	The data fits comfortably in RAM with room for intermediates — in practice under a few GB. Interactive, exploratory, iterative work. Fastest path to an answer, by a wide margin.
Hadoop MapReduce	The data exceeds one machine's memory or disk; the job must survive node failure without restarting from scratch; the work is a scheduled batch pass over historical data where a fixed one-minute overhead is irrelevant; or the volume is growing and the architecture must scale by adding machines rather than by buying a bigger one.

For this specific quarter of taxi data on this specific machine, Pandas is the correct engineering choice, and the honest conclusion of the benchmark is to say so. The reason to build the Hadoop pipeline is that the same code, unchanged, runs against ten years of data on a fifty-node cluster — and the Pandas script does not run against it at all.

One caveat in Hadoop's favour that Table 12.1 understates: the cleaning job (Section 6) shuffled a full gigabyte to deduplicate 11,077,209 records. A Pandas equivalent would need every record resident simultaneously to detect duplicates, at roughly 1.3 GB per gigabyte of input. The distributed version has no such requirement, because the deduplication happens inside a sort that is allowed to spill to disk.

13. Business Insights

The twelve questions posed by the case study, answered from the results above.

(a) What is the busiest hour for taxi demand?

18:00, with 696,255 trips — 6.63% of the quarter. The quietest is 04:00 with 89,214. More useful than the peak itself is the plateau: every hour from 12:00 to 22:00 exceeds half a million trips, so there is no single hour worth optimising for.

(b) What is the busiest day of the week?

Saturday (1,726,797 trips); the quietest is Monday (1,184,386). Per day, the weekend averages 1,540,734 trips against 1,482,490 on weekdays — 3.9% busier, which the raw weekday-versus-weekend totals hide.

(c) Which pickup zones generate the most trips?

Upper East Side South (Manhattan), Upper East Side North (Manhattan), Midtown Center (Manhattan) lead, and the top ten zones account for 33.4% of all 10,493,916 trips. All ten are Manhattan zones or airports.

(d) Which pickup zones generate the most revenue?

A different list. JFK Airport (Queens) is first with \$32,399,670 despite ranking only fourth by trip count, followed by LaGuardia Airport (Queens) (\$17,322,968). The two airports alone contribute \$49,722,638 — 15.7% of all revenue — from 6.2% of the trips.

(e) Which payment method contributes the most revenue?

Credit card: \$199,656,180, or 63.2% of the total, across 6,739,968 trips. Cash has fallen to 7.0%.

(f) Do credit-card users generate more tips?

In the data, overwhelmingly: card trips average \$4.13 in tips with 90.4% of trips tipping, against effectively zero for cash.

But this comparison is not valid as a statement about customer behaviour. The meter records a tip only when it is charged to the card; a cash tip never enters the dataset. The defensible claim is the narrow one — among card payers, 90.4% tip at 21.1% of fare. Any statement about whether cash customers tip requires data this system does not collect.

(g) What distance category produces the highest average fare?

The 20-plus mile band, at \$90.26 per trip — but it is only 0.85% of trips. Per mile the ranking inverts: 0–2 mile trips earn \$10.55 per mile against \$3.62 for the longest band. Long trips are the biggest tickets; short trips are the best rate.

(h) What are the most frequently travelled routes?

Upper East Side South -> Upper East Side North (68,497); Upper East Side North -> Upper East Side South (58,863); Upper East Side North -> Upper East Side North (46,029). All are short intra-Manhattan hops averaging around a mile.

(i) Are the most frequent routes also the most profitable?

No, and this is the most actionable finding in the report. Only 7 routes appear in both the top 20 by trips and the top 20 by revenue.

The busiest route (Upper East Side South -> Upper East Side North) earned \$1,107,952 from 68,497 trips. The highest-earning route (JFK Airport -> Outside of NYC) earned \$2,403,940 from just 19,520 — 8 times the revenue per trip. Optimising dispatch on frequency would send vehicles to the wrong side of the city.

(j) What percentage of records contain potential anomalies?

31.01% of the raw feed triggered at least one detector (3,434,517 records). That figure needs decomposing before it is quoted: 28.0% is a missing `passenger_count`, which is a blank field rather than a bad trip. Excluding it, genuinely defective records are about 4% — dominated by zero distances (3.34%) and negative amounts (0.80%), the latter being refund rows rather than errors.

(k) What transportation insights can management derive?

Position for the evening, not the morning. There is no morning commuter peak; demand builds through the afternoon and holds from 12:00 to 22:00. Shift patterns built around a split morning/evening rush are mismatched to the data.

Treat the airports as a separate business. JFK and LaGuardia produce 15.7% of revenue from a small minority of trips, at average fares of \$61.79 and \$44.49. Airport queue management deserves its own operating policy.

Manage the two products differently. Short Manhattan hops are a high-frequency, high-rate-per-mile, low-speed product; airport and outer-borough runs are low-frequency, high-ticket, high-speed. They have different economics and should not share a single utilisation target.

Watch the FlexFare channel. It is already 29.1% of revenue at the highest average fare of any payment type, with almost no recorded tips. Whatever it represents, driver earnings in that channel are not visible in this data.

Fix `passenger_count` at source. A quarter of the feed is missing it. Any future analysis of occupancy or vehicle sizing is impossible until that collection gap is closed.

(l) When does Hadoop MapReduce provide a meaningful advantage?

Not here. At 1.03 GB on one machine, Pandas finished in 11.0 seconds against Hadoop's 192 — 18 times faster — and the honest recommendation for this quarter of data is to use Pandas.

The advantage appears at three thresholds. First, when the data no longer fits in memory: the full-load Pandas variant needed 1,347 MB for 1.03 GB, so an 8 GB machine fails at roughly one year of this feed. Second, when the job must be reliable: a mid-job node failure re-runs one task in MapReduce and restarts the whole script in Pandas. Third, when scale must be bought with machines rather than with a bigger machine — Hadoop's 15.2 s per million records is per node and divides across the cluster; Pandas has no such term.

The 32-second fixed overhead measured in Section 12.2 is what makes Hadoop the wrong tool for interactive work and an entirely reasonable one for a scheduled nightly pass over years of history.

14. Limitations

14.1 Of the platform

- **Single node.** A pseudo-distributed cluster demonstrates every mechanism — block splitting, containers, shuffle, fault handling — but it cannot demonstrate the benefit, because there is no second node for a task to move to. All timings in Section 12 are single-node timings and should be read as an upper bound on Hadoop's cost, not as a measure of its scalability.
- **Two physical cores.** YARN was configured for 4 vcores, so the configured parallelism exceeded the real parallelism and containers contended for CPU.
- **Replication factor 1.** With one DataNode, HDFS's fault tolerance is configured but not exercised. Losing the DataNode would lose the data.
- **No native libraries.** Java implementations of checksumming and compression were used throughout, which penalises the Hadoop side of the benchmark by an unmeasured amount.
- **Python startup cost.** Hadoop Streaming forks a Python process per task and serialises every record to text across a pipe. A native Java MapReduce job, or a Spark job over the original Parquet, would avoid both costs — at the price of the readability that made this project practical.

14.2 Of the data

- **Cash tips are invisible.** The single most important caveat in the report. Every tipping comparison between payment types is a comparison of what the meter records, not of what customers do.
- **passenger_count is 25.5% missing.** Retained and flagged rather than deleted (Section 6.3), but no occupancy analysis is possible.
- **One quarter, one taxi class.** January–March 2026 only, yellow taxis only. No seasonal comparison is possible, and no conclusion here extends to green taxis or for-hire vehicles, which serve substantially different geographies.
- **Zones, not coordinates.** The TLC publishes 263 zone identifiers rather than latitude and longitude, so route analysis is at neighbourhood granularity. Two trips within one large zone are indistinguishable from each other.
- **FlexFare is undocumented here.** payment_type 0 carries 2,784,548 trips in this feed. Its exact meaning was not verified against a TLC data dictionary, and the interpretation offered in Section 9.5 is an inference from its fare and tip profile, not a documented fact.

14.3 Of the method

- **Thresholds are judgements.** 200 miles, 24 hours, \$1,000, 100 mph — each is defensible and none is derived from the data. A different analyst would choose differently and would get slightly different totals. The counters make the sensitivity checkable, which is the point of recording them.
- **Revenue is total_amount.** It includes tolls, surcharges and taxes, so it is gross receipts rather than driver or operator earnings.
- **Deduplication is exact-match only.** Two records differing by one cent or one second are treated as distinct trips. A fuzzy near-duplicate detector would need a different key design entirely — for instance locality-sensitive hashing — and would be a considerably harder MapReduce problem.

15. Conclusion

A complete Hadoop analytics pipeline was designed, implemented and executed over 11,077,209 NYC yellow-taxi trip records. HDFS stored the 1.1 GB dataset across ten 128 MB blocks; a two-phase MapReduce job validated and deduplicated it down to 10,493,916 analysable records; nine analytical jobs and five Top-N jobs, all written as Python programs run through Hadoop Streaming, produced the results in Section 9; and YARN scheduled all 18 applications, every one of which succeeded.

The analysis itself produced three findings a trip-count summary would have missed. Demand is a broad afternoon-to-evening plateau rather than a commuter peak, and the weekend is busier per day than the

weekday once the totals are normalised. Revenue and volume point at different parts of the city — the two airports produce 15.7% of revenue from a small fraction of trips. And only 7 of the top twenty routes by frequency also appear in the top twenty by revenue, which means any dispatch policy tuned on trip counts is optimising the wrong quantity.

On the technical side, the exercise made three things concrete that are easy to state and hard to feel. The choice of map output key is the algorithm — a composite key gives a two-column GROUP BY, the whole record as key gives a distributed DISTINCT, and a constant key gives a global ranking at the cost of a serial reduce. Averages are not associative, which is why every mapper here emits sums and counts and why one generic combiner could serve five different analyses, cutting the route job's shuffle by a factor of 41. And ranking genuinely cannot be done in a single job, which is why the multi-stage workflow exists for a reason rather than for a mark.

The benchmark is the part worth being blunt about. Pandas beat Hadoop by a factor of 18 on this dataset, and for a single quarter of taxi data on a single machine, Pandas is the right tool. Hadoop's 32-second fixed overhead is unrecoverable at this scale. What the three-point scaling measurement shows is where that stops being true: the ratio narrows as data grows, Pandas' memory ceiling arrives at roughly one year of this feed on an 8 GB machine, and Hadoop's per-record cost is the only one of the two that divides by the number of nodes. The pipeline built here would run unchanged over ten years of data on fifty machines. The Pandas script would not run over it at all.

That is the real lesson of the assignment, and it is not "Hadoop is fast". It is that the interesting property of a distributed system is not its speed on data that fits, but its behaviour on data that does not.

16. References

- Apache Software Foundation. Apache Hadoop 3.5.0 Documentation — HDFS Architecture. <https://hadoop.apache.org/docs/r3.5.0/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>
- Apache Software Foundation. Hadoop Streaming. <https://hadoop.apache.org/docs/r3.5.0/hadoop-streaming/HadoopStreaming.html>
- Apache Software Foundation. Apache Hadoop YARN. <https://hadoop.apache.org/docs/r3.5.0/hadoop-yarn/hadoop-yarn-site/YARN.html>
- Apache Software Foundation. MapReduce Tutorial. <https://hadoop.apache.org/docs/r3.5.0/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html>
- Dean, J. and Ghemawat, S. (2004). MapReduce: Simplified Data Processing on Large Clusters. OSDI 2004.
- Ghemawat, S., Gobioff, H. and Leung, S.-T. (2003). The Google File System. SOSP 2003.
- White, T. (2015). Hadoop: The Definitive Guide, 4th edition. O'Reilly Media.
- Leskovec, J., Rajaraman, A. and Ullman, J. (2020). Mining of Massive Datasets, 3rd edition. Cambridge University Press. Chapter 2, MapReduce design patterns.
- NYC Taxi & Limousine Commission. TLC Trip Record Data. <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
- NYC Taxi & Limousine Commission. Yellow Trips Data Dictionary and Taxi Zone Lookup Table.
- Apache Software Foundation. Apache Parquet Documentation — File Format. <https://parquet.apache.org/docs/file-format/>

Appendix — Reproducing this work

The complete project accompanies this report:

- `commands.txt` — every HDFS and Hadoop Streaming command, in execution order.
- `README.md` — environment assumptions, project layout and execution instructions.
- `mappers/` and `reducers/` — all 22 Python programs.
- `evidence/` — one driver log per job, the counter dumps, and the HDFS/YARN evidence files E1–E6.
- `output/` — the reducer output behind every table and figure in this report.
- `charts/` — the seven figures, generated by `scripts/make_charts.py` from `output/`.